Constraint-Aware Causal LM Fine-Tuning with Serialization-Focused Continuation, Deterministic Post-Generation Repair, and Hybrid Selection for Text-to-SVG

Nidhish Gautam (ng3483) and **Dheeraj Pakala (dp4086)**
New York University (NYU)

Abstract

We present a practical, constraint-aware pipeline for text-to-SVG generation under strict submission rules. Our approach uses a causal language model (Qwen2.5-3B) fine-tuned with a serialization-focused continuation strategy to improve completion of final SVG outputs. While raw generations often contain truncation and formatting errors, we show that many failures are near-valid and can be recovered with deterministic post-generation repair. We combine model outputs with rule-based validation, lightweight structural fixes, and hybrid row-level fallback selection to maximize final submission reliability. On a 1,000-prompt evaluation, raw valid XML before repair is substantially lower than final compliant output, highlighting the importance of integrating modeling with constraint-aware post-processing. The resulting system emphasizes robustness over single-pass generation purity and provides a reproducible recipe for competitive text-to-SVG submission pipelines.

1 Introduction

Text-to-SVG generation is challenging because outputs must be both semantically correct and structurally valid programs. In competition settings, this is further constrained by strict submission rules (e.g., XML well-formedness, allowed tags, bounded complexity, and fixed output schema). As a result, the task is not only prompt-to-graphics generation, but also reliability engineering under hard validator constraints.

Our primary pipeline is generation-first: we use a constraint-aware causal language model (Qwen2.5-3B) fine-tuned for SVG serialization quality, including a serialization-focused continuation stage to reduce truncation and malformed outputs. This model handles the majority of examples, followed by deterministic validation and repair to enforce submission compliance.

For difficult cases, we introduce a retrieval-based fallback. Rather than invoking retrieval for every prompt, we trigger it selectively when generation confidence is low or when prompts exhibit hard characteristics (e.g., logo-like intent, number-heavy symbols, or high compositional complexity). The retrieval module normalizes prompts, extracts visual keywords and lightweight intent signals, and scores candidate training examples using a combination of embedding similarity, keyword overlap, and intent-aware re-ranking. When a strong candidate is found, we optionally apply conservative adaptation (such as limited color correction), then run the same validation and repair checks.

This design keeps generation as the default path while using retrieval strategically as a robustness mechanism for failure-prone examples. Final submissions are produced through strict validation, deterministic repair, and row-level hybrid selection. Overall, the system combines generative flexibility with targeted retrieval support to balance quality, validity, and efficiency in competition-grade text-to-SVG pipelines.

2 Method

2.1 Problem Setup and Constraints

We address text-to-SVG generation under strict competition constraints. Given a prompt x, the system must generate an SVG output y such that:

1. y is well-formed XML with `<svg>` as the root element,
2. y satisfies validator constraints (allowed tag set, bounded path count, bounded output length, and fixed submission schema), and
3. y preserves the prompt-conditioned visual semantics.

Thus, the task is not unconstrained text generation; it is constrained program generation with submission-time validity requirements.

3 Methodology

3.1 Data Preprocessing and Canonicalization

Before training, we normalize raw SVG data using canonicalization and safe canonicalization. We measure target-format variance using root-level serialization signatures (unique `<svg ...>` attribute-order patterns). Canonicalization reduces this count from 115 to 48 (**58.3%**), and safe canonicalization reduces it to 47 (**59.1%**), yielding more consistent attribute ordering and cleaner target formatting. The XML parse-fail rate remains 0% before and after canonicalization, while SVG length variance is reduced by 0.33% (canonical) and 0.75% (safe canonical). All constraint-aware normalization is applied to training SVG targets only; no test-time checker outputs or labels are used for training supervision.

Figure 1: Example from our preprocessing pipeline showing the same sample before canonicalization (Original), after canonicalization, and after safe canonicalization.

Feature-engineered training schema We build an enriched training table (`final_training.csv`) with both structured and validator-aware signals. We also generate deterministic schema previews to verify column integrity and serialization consistency before training, reducing silent data-schema failures. Each feature is computed from canonicalized training targets and scene structure metadata to provide stable supervision signals for composition, geometry, compactness, and validator-facing correctness. Table 1 summarizes the engineered schema.

Field(s)	Purpose
`layout_target`, `detail_target`, `serialization_target`	Staged supervision for global composition, object-level details, and final deterministic SVG serialization.
`scene_version`	Tracks canonicalized scene representation variant for reproducibility and consistency.
`object_count`	Supervises scene complexity and expected number of visual elements.
`validity_target`	Provides explicit compliance supervision for validator-facing correctness.
`compactness_target`	Encourages concise SVG programs under length/complexity constraints.
`visibility_ratio`	Captures how much generated content remains visually effective on canvas.
`overlap_graph`	Encodes inter-object spatial overlap relationships for structural reasoning.
`path_command_histogram`	Summarizes geometric command usage patterns for shape/complexity control.
`structure_proxy_sequence`	Supplies lightweight sequential structure signals complementary to raw SVG text.
`fold_id`	Identifies data split membership for robust training/evaluation partitioning.

Table 1: Fields in `final_training.csv` and their roles in structure, compliance, and serialization supervision.

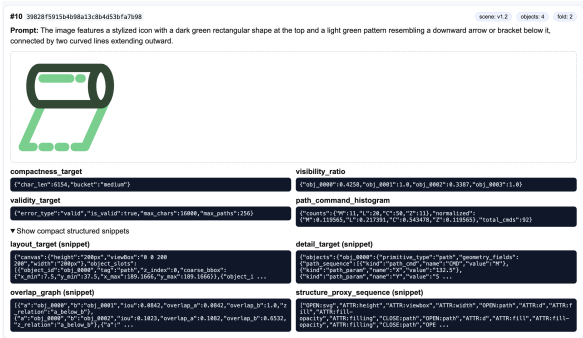

Figure 2: Feature-engineered training sample preview. Example row from `final_training.csv` showing prompt, rendered SVG, and engineered feature fields used for staged supervision and constraint-aware training.

3.2 Training Methodology

Our training follows a phased schedule to balance semantic learning and constraint compliance. We use mixed-precision (`bf16`) multi-GPU fine-tuning of a causal LM.

Training setup Our training stack is based on PyTorch and Hugging Face Transformers, executed in distributed multi-GPU mode. We run autoregressive fine-tuning for three epochs in total (Phase 1–2 standard fine-tuning, Phase 3 serialization-focused continuation), with checkpointing at intermediate steps for model selection. Data loading is driven by the feature-engineered training table (`final_training.csv`), and training examples are serialized into the staged target contract before tokenization. To keep runs reproducible and robust, we use deterministic preprocessing, fixed target formatting, and a resume-friendly training workflow.

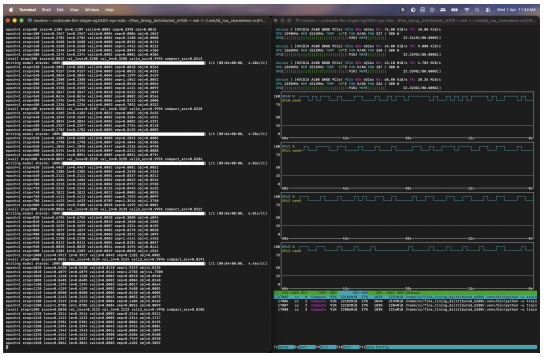

Figure 3: We used a PyTorch + Hugging Face (Transformers/Accelerate) stack with mixed-precision bf16, distributed multi-GPU training/inference via torchrun (NCCL backend) on A100-class GPUs, and a Kaggle single-GPU inference variant for final submission generation.

Training instance format Each supervised sample is serialized as a feature-conditioned training record with staged fields in a fixed order: `layout_target` → `detail_target` → `serialization_target`. This consistent contract reduces formatting entropy and keeps optimization focused on predictable structure boundaries. In practice, the model learns both semantic mapping (prompt → scene structure) and deterministic string realization (scene structure → SVG text).

Structured supervision (DSL-style staged targets) Instead of supervising only raw final SVG text, we use a staged target representation:

- `layout_target`: global composition/canvas and object-slot structure,
- `detail_target`: object-level geometry and style fields,
- `serialization_target`: deterministic final SVG text.

This DSL-style decomposition separates high-level structure from final serialization while preserving the final SVG contract required for submission. It also provides intermediate supervision signals that make failure modes easier to localize during error analysis. Despite staged supervision, truncation remains a practical failure mode before complete `serialization_target` closure; in a 1000-row run, only 346 rows are valid XML before repair (34.6%).

Phase 1–2: standard fine-tuning In the first two epochs, we perform standard supervised autoregressive fine-tuning on feature-engineered training records with next-token prediction. Each record includes the prompt, staged targets (`layout_target`, `detail_target`, `serialization_target`), and auxiliary structured fields used in our schema. During this stage, all staged targets participate in learning, so the model first stabilizes prompt-to-structure and prompt-to-geometry alignment before strict serialization refinement.

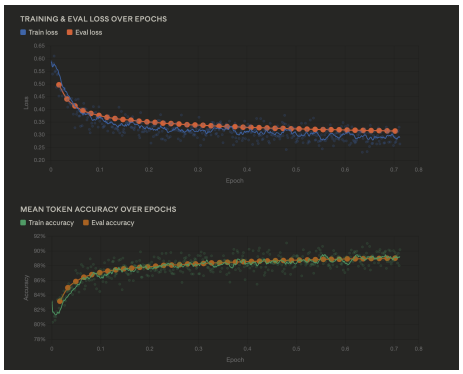

Figure 4: Phase 1–2 standard fine-tuning overview.

Phase 3: constraint-aware continuation In the final epoch, we shift to a constraint-aware serialization-focused continuation stage. This phase prioritizes completion of valid `serialization_target` outputs and reduces truncation-driven submission failures. Operationally, supervision is restricted to serialization behavior, with auxiliary objectives disabled in the

strict run. Concretely, label supervision is concentrated on serialization-relevant tokens, so optimization pressure is directed at closing tags, maintaining valid XML structure, and reducing incomplete endings.

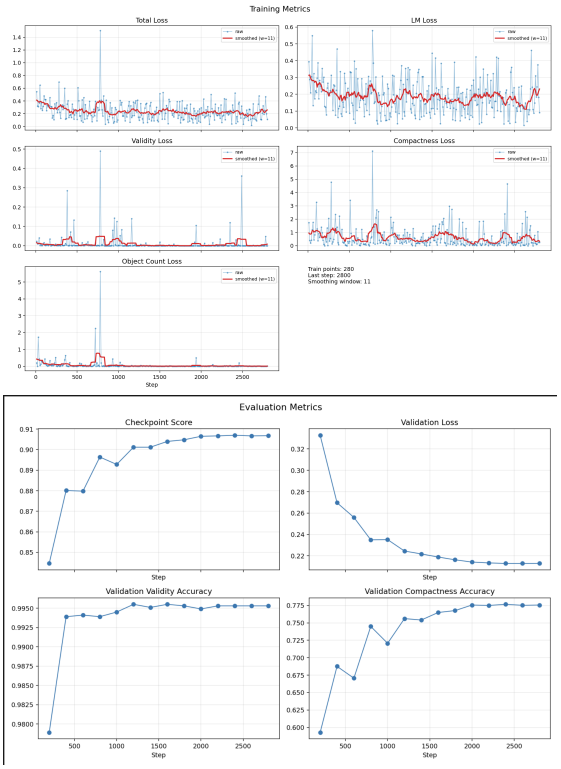

Figure 5: Phase 3 constraint-aware continuation training metrics (two runs/panels).

3.3 Inference Pipeline

At inference time, we prioritize determinism, resumability, and clean handoff to downstream repair. We run decoding in deterministic (greedy) mode with a fixed token budget (`max_new_tokens`) to reduce formatting variance across runs. When structured prompting is enabled, generation is anchored to the staged contract and serialization-prefill behavior so the model is encouraged to continue directly in the target SVG-oriented serialization format.

Distributed and resume-safe execution Inference is executed in distributed multi-GPU mode, where each rank processes a disjoint shard of the test set. Each rank writes outputs to its own shard CSV file with atomic append-style updates, enabling interruption-safe progress. Before processing, each rank loads already completed IDs from its shard and skips them, so reruns resume from unfinished examples instead of restarting the full split. After all ranks finish, rank 0 performs a deterministic merge of per-rank shards into a single raw generations file.

Raw output extraction Generated continuations are stored as raw text first, then parsed using marker-aware extraction to isolate the serialization payload. To handle prompt/template drift, extraction supports multiple marker variants (e.g., `serialization_target:`, `serialization:`, `SVG target:`, and fallback markers) and selects the earliest reliable match. This step produces a canonical raw generations artifact for downstream deterministic repair and validation.

3.4 Deterministic Repair and Validation

Because many raw outputs are near-valid but structurally incomplete, we apply a deterministic repair pass before submission assembly. The repair stage is rule-based and fully reproducible: it crops to plausible SVG regions, strips prompt leakage/noise, recovers dangling shape-tag tails, trims to the last complete tag boundary, and enforces a valid closing structure. We then parse candidates as XML and retain repaired outputs that satisfy structural checks.

Validator-facing checks Post-repair validation enforces XML well-formedness, expected root structure, and submission-oriented constraints. In addition to validity, we track a graphics-content signal (presence of drawable primitives) to separate formally valid-but-empty outputs from practically useful rows. This yields both repaired SVG strings and audit metadata (repair status, validity flags, graphics flags) used by later selection stages.

3.5 Hybrid Selection and Fallback Strategy

No single generation pass is uniformly best across all prompts, so final submission construction uses row-level hybrid selection with a retrieval-assisted fallback tier. Our primary source is repaired model output; fallback rows are filled from a secondary source when the primary row fails strict checks or is judged low-confidence. Selection is deterministic and ID-aligned, ensuring exactly one SVG per required test ID with no duplication.

Retrieval preprocessing and feature extraction Before retrieval, prompts are cleaned by removing the fixed instruction prefix/suffix template so similarity is computed on the semantic core description

only. For all training SVGs, we precompute two retrieval-side structural features: `has_fill` (filled vs. stroke-only behavior) and `n_paths` (shape-count proxy for visual complexity). We also extract prompt keywords from a curated 68-term visual lexicon and compute keyword-overlap similarity using Jaccard score.

Intent extraction and routing priors For each test prompt, we compute binary intent flags: `is_outline`, `is_logo`, `has_number`, `is_complex`, and `wants_fill`. Prompts marked logo-like, number-sensitive, or highly complex are forced to use the LLM path via `must_llm`, because direct retrieval reuse is brittle for these cases. These intent signals are also used as re-ranking bonuses/penalties to prefer stroke-like or filled candidates when requested.

Retrieval preprocessing flow:
Raw prompt → Prompt cleaning → Clean prompt text
Clean prompt text → Keyword extraction (68-term lexicon) → Keyword-overlap signal (Jaccard)
Training SVG pool → Precompute `has_fill`, `n_paths`
Clean prompt text → Intent extraction (`is_outline`, `is_logo`, `has_number`, `is_complex`, `wants_fill`)
Intent flags → `must_llm` prior and re-ranking bonuses/penalties
Combined signals → Retrieval scoring → Routing decision

Figure 6: Compact diagram of retrieval preprocessing, intent extraction, and routing priors.

Dual-encoder retrieval and intent-aware re-ranking Candidate retrieval uses two sentence-transformer encoders with weighted cosine fusion: `all-MiniLM-L6-v2` (weight 0.35) and `all-mpnet-base-v2` (weight 0.65), with L2-normalized embeddings. For each test row, we select top-k candidates by fused similarity ($k = 5$), then apply composite intent-aware re-ranking:

$$s = 0.93 \cdot e + 0.07 \cdot \mathrm{Jaccard}_{\mathrm{kw}} + \mathrm{bonus},$$

where e is dual-encoder cosine similarity, $\mathrm{Jaccard}_{\mathrm{kw}}$ is keyword overlap, and bonus encodes intent-specific preferences (fill/stroke match and small complexity bonus from path count). The highest-scoring candidate provides the Tier-1 retrieval SVG and confidence score. These encoders are retrieval/ranking utilities only; they do not generate SVG outputs.

Tier routing and conservative recoloring Tier-2 routing is triggered when `must_llm` is true or retrieval confidence falls below threshold (`sim` < 0.88). In Tier-2, SVG generation is performed by the same fine-tuned Qwen2.5-3B model used throughout this work (no additional generative model is introduced). Before finalizing retrieval output, we optionally apply conservative color correction: recoloring is allowed only when the prompt is not outline-only, the candidate is not stroke-only, the prompt explicitly requests chromatic colors, and the dominant candidate color family conflicts with prompt intent. This prevents over-aggressive edits while still correcting obvious color-family mismatch.

Final assembly After hybrid selection, we run one final compliance sweep over the full submission file. This last pass verifies schema-level consistency (`id`, `svg`), uniqueness/coverage of IDs, and validator-facing SVG constraints before export. The resulting pipeline converts partially reliable raw generation into a robust, competition-ready submission artifact.

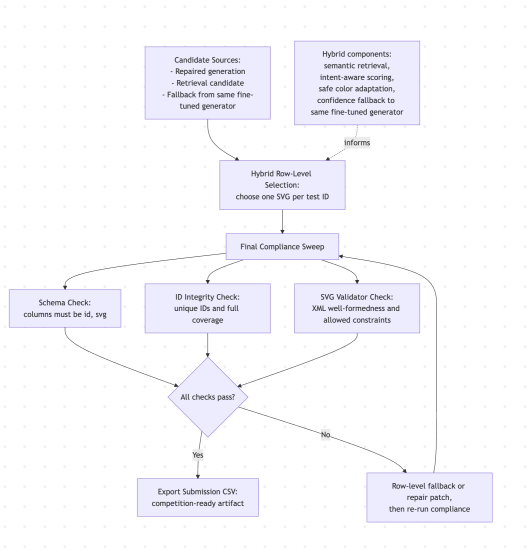

Figure 7: End-to-end final assembly and hybrid selection pipeline.

In summary, the hybrid stage combines semantic retrieval, intent-aware scoring, safe color adaptation, and confidence-based fallback to the same fine-tuned generator to improve reliability on hard prompts without sacrificing deterministic submission behavior.

4 Experiments and Results

4.1 Experimental Setup

We evaluate on a 1,000-prompt test subset, using the same submission-oriented pipeline described in Section 3. Inference is run with deterministic greedy decoding and a fixed generation budget (`max_new_tokens`), so output variability is controlled across reruns. Evaluation is reported in three stages: (i) raw generations before repair, (ii) deterministic repair output, and (iii) final hybrid submission after fallback-based row selection and final compliance checks.

4.2 Main Quantitative Results

Table 2 summarizes the primary outcomes. Before repair, only 346/1000 generations are valid XML (34.6%), even though 347/1000 rows contain a full `<svg ... </svg>` span. This confirms that many failures are near-valid but still rejected by strict parsing/structure checks. Applying minimal deterministic repair recovers 651 additional rows. After hybrid selection and final validation, the final submission is fully valid with complete ID coverage (1000/1000).

Metric	Value
Raw valid XML before repair	346/1000 (34.6%)
Rows with full `<svg ... </svg>` block	347/1000
Additional rows recovered by minimal deterministic repair	651
Final hybrid submission validity	1000/1000

Table 2: Main quantitative results across raw generation, repair, and final hybrid submission.

4.3 Ablation / Contribution Analysis

We analyze the contribution of each pipeline stage under a fixed 1,000-prompt setting and identical validator checks. Table 3 reports cumulative validity as components are added. The largest gain comes from deterministic repair, confirming that most failures are near-valid structural errors rather than complete semantic misses. Hybrid selection then closes the remaining gap to full coverage by replacing unresolved rows with validated fallback outputs.

4.4 Alternative Methods and Iterative Improvements

Our final system emerged from several intermediate approaches that were useful for diagnosing failure modes. First, we tested few-shot prompting on the base (non-fine-tuned) model. Although this improved local formatting in some rows, it remained unreliable under strict validator constraints and often failed to produce submission-ready SVG at scale.

Pipeline Variant	Valid XML
Raw generation only (no repair, no hybrid)	346/1000 (34.6%)
+ Deterministic repair	997/1000 (99.7%)
+ Hybrid fallback selection (final system)	1000/1000 (100.0%)

Table 3: Ablation-style cumulative contribution of major pipeline components.

Second, we trained without canonicalization. This setting increased target-format variance and led to poorer serialization behavior, with more malformed endings and lower consistency in generated SVG structure. Empirically, non-canonicalized training produced weaker SVG validity and was not retained.

Third, we evaluated an early fine-tuned model after two epochs. This model improved semantic alignment relative to base prompting, but complete SVG closure remained weak: fewer than 10% of rows produced full `<svg ... </svg>` outputs in our early checks. This exposed serialization completion—not semantic grounding—as the dominant bottleneck.

We then introduced a one-run constraint-aware continuation stage focused on serialization quality. This produced the strongest raw generations among all training variants, with substantially more structurally correct outputs, though many still required deterministic repair. After repair, we applied our final hybrid strategy (retrieval-assisted fallback plus strict final validation), yielding robust end-to-end submission quality.

5 Limitations and Failure Cases

Despite strong submission validity, the pipeline still has important limitations. First, semantic fidelity is not guaranteed by XML validity: some repaired outputs are structurally correct but visually simplistic or only loosely aligned with prompt intent. Second, retrieval can fail when lexical overlap is misleading or when a prompt describes uncommon compositions; in these cases, high similarity scores may still return semantically mismatched exemplars. Third, deterministic repair is intentionally

conservative and syntax-oriented, so it can recover structure but cannot reliably invent missing semantic detail after severe truncation. As a result, the system is strongest at converting near-valid generations into compliant SVGs, but weaker on truly hard prompts requiring precise symbolic or compositional reasoning.

6 Conclusion

We presented a constraint-aware text-to-SVG pipeline that combines fine-tuned causal generation with deterministic repair and hybrid fallback selection. On a 1,000-prompt evaluation, raw valid XML before repair is 346/1000 (34.6%), deterministic repair raises validity to 997/1000 (99.7%), and final hybrid assembly achieves 1000/1000 submission-valid rows. These results show that submission robustness is driven not only by model quality, but by end-to-end systems design spanning preprocessing, constrained training, repair, and final compliance checks. Our experiments also indicate that canonicalization and serialization-focused continuation are key to reducing truncation-related failures. Overall, the approach provides a practical and reproducible recipe for competition-grade SVG generation under strict validator constraints.

References (Key Works)

- Carlier, A., Danelljan, M., Alahi, A., Timofte, R., and Van Gool, L. 2020. *DeepSVG: A Hierarchical Generative Network for Vector Graphics Animation*. NeurIPS.
- Reimers, N. and Gurevych, I. 2019. *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. EMNLP-IJCNLP.
- Song, K., Tan, X., Qin, T., Lu, J., and Liu, T.-Y. 2020. *MPNet: Masked and Permuted Pre-training for Language Understanding*. NeurIPS.
- Post, M. and Vilar, D. 2018. *Fast Lexically Constrained Decoding with Dynamic Beam Allocation for Neural Machine Translation*. NAACL.
- Lewis, P., Perez, E., Piktus, A., et al. 2020. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS.